Postgres Concurrency, Transactions & the Outbox Pattern in Dentacore

Nov 9, 2026 · @Nam

Overview

Dentacore guards invariants in Postgres first (constraints, guarded updates), and delivers side effects like email through a transactional outbox (`@nestjs/outbox`), not a direct call after commit.

Key decisions:

- **Prisma Client by default, raw SQL only when Prisma can't do it:** row locks, advisory locks, range overlap, reporting.
- **Invite race:** fixed with a guarded `updateMany` / `update` on `inviteStatus` + `deletedAt`, not with Serializable.
- **Email delivery:** the status change and the outbox message commit in one transaction; a handler creates the token and sends the email, retried with backoff.
- **Invite token:** created in the handler (raw token never stored), sent in a `#fragment` link, posted back in the request body.
- **Transactions:** `@nestjs-cls/transactional`; anything that must run after commit (e.g. `outbox.notify()`) sits outside the transactional method.
- **No RabbitMQ for now:** the outbox's local transport delivers to in-process handlers.

Transactions and isolation levels

Every statement already runs in an implicit transaction; an explicit one matters when several statements must succeed together or see one consistent snapshot.

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;            -- or ROLLBACK;

BEGIN ISOLATION LEVEL REPEATABLE READ;   -- level set before the first query

-- Savepoints
BEGIN;
INSERT INTO patients (name) VALUES ('A');
SAVEPOINT before_b;
INSERT INTO patients (name) VALUES ('B');
ROLLBACK TO SAVEPOINT before_b;   -- A is kept
COMMIT;

BEGIN TRANSACTION READ ONLY;

SET lock_timeout = '3s';
SET idle_in_transaction_session_timeout = '30s';
```

Level	Non-repeatable read	Phantom	Lost update	Write skew
Read Committed (default)	possible	possible	possible	possible
Repeatable Read (snapshot)	prevented	prevented	errors with 40001	possible
Serializable (SSI)	prevented	prevented	prevented	prevented

Postgres never allows dirty reads; Read Uncommitted behaves as Read Committed.

When reads belong in a transaction

Situation	Transaction?
One standalone read for display	No
Read that decides a write	Yes, same transaction as the write, plus a lock, guard or constraint

Situation	Transaction?
Several reads that must agree (count + page, report)	Yes, Repeatable Read, read-only
Several writes that succeed or fail together	Yes

Wrapping every read costs pool connections and long transactions block `VACUUM`. Keep HTTP calls and emails outside transactions.

Race condition types

Six anomalies matter in Postgres; the double booking (phantom) and write skew are the ones a transaction alone does not stop.

Race	What goes wrong	Dentacore example
Lost update	Two read-modify-writes; the second overwrites the first	Stock decremented once instead of twice
Non-repeatable read	The same row read twice in one transaction changes in between	Appointment status flips mid-transaction
Phantom / check-then-insert	Both check "no row exists", both insert; `FOR UPDATE` can't lock a missing row	Dentist double-booked
Write skew	Both read the same set, each writes a different row, the combined result breaks a rule	Last two on-call dentists both go off call
Duplicate insert	Both check an email is free, both insert	Two patients with one email
Deadlock	Each transaction waits on a lock the other holds; one is aborted (40P01)	Two transfers locking rows in opposite order

```
-- Lost update
-- S1: SELECT stock FROM items WHERE id=1;   -- 10
-- S2: SELECT stock FROM items WHERE id=1;   -- 10
-- S1: UPDATE items SET stock = 9 WHERE id=1; COMMIT;
```

```
-- S2: UPDATE items SET stock = 9 WHERE id=1; COMMIT;   -- should be 8

-- Phantom: both sessions run this concurrently, both see 0, both insert
BEGIN;
SELECT count(*) FROM appointments
WHERE dentist_id = 3
  AND tstzrange(starts_at, ends_at) && tstzrange('2026-11-10 09:00+07',
'2026-11-10 09:30+07');
INSERT INTO appointments (dentist_id, patient_id, starts_at, ends_at)
VALUES (3, 42, '2026-11-10 09:00+07', '2026-11-10 09:30+07');
COMMIT;

-- Write skew (Repeatable Read does NOT catch this)
-- S1 + S2: BEGIN ISOLATION LEVEL REPEATABLE READ;
-- S1 + S2: SELECT count(*) FROM dentists WHERE on_call;          -- 2
-- S1: UPDATE dentists SET on_call=false WHERE id=1; COMMIT;
-- S2: UPDATE dentists SET on_call=false WHERE id=2; COMMIT;     -- 0 on call
```

Mitigation approaches

Layer them: constraints for every invariant you can declare, atomic updates for read-modify-write, and Serializable or `FOR UPDATE` for cross-row rules constraints can't express.

Race	First choice	Alternatives
Lost update	Atomic `UPDATE ... SET x = x - 1`	`FOR UPDATE`, version column, Repeatable Read + retry
Non-repeatable read	Repeatable Read	`FOR SHARE`
Phantom / double booking	Exclusion constraint	Lock a parent row, advisory lock, Serializable
Write skew	Serializable + retry	`FOR UPDATE` on every row the decision reads
Duplicate insert	`UNIQUE` + `ON CONFLICT`	Advisory lock
Deadlock	Consistent lock order	`lock_timeout`, short transactions, retry on 40P01

```
-- 1. Atomic guarded update
UPDATE items SET stock = stock - 1 WHERE id = 1 AND stock >= 1 RETURNING
stock;

-- 2. Pessimistic lock
BEGIN;
SELECT stock FROM items WHERE id = 1 FOR UPDATE;   -- others wait
UPDATE items SET stock = stock - 1 WHERE id = 1;
COMMIT;
-- variants: FOR NO KEY UPDATE | FOR SHARE | FOR KEY SHARE | NOWAIT | SKIP
LOCKED

-- Job queue
SELECT id, payload FROM jobs WHERE status = 'pending'
ORDER BY created_at LIMIT 1 FOR UPDATE SKIP LOCKED;

-- 3. Optimistic lock (version column)
UPDATE patients SET phone = '0412...', version = version + 1
WHERE id = 42 AND version = 7 RETURNING version;   -- 0 rows = conflict

-- 4. Constraints
ALTER TABLE patients ADD CONSTRAINT patients_email_key UNIQUE (email);
ALTER TABLE items ADD CONSTRAINT stock_non_negative CHECK (stock >= 0);

CREATE EXTENSION IF NOT EXISTS btree_gist;
ALTER TABLE appointments ADD CONSTRAINT no_dentist_overlap
EXCLUDE USING gist (dentist_id WITH =, tstzrange(starts_at, ends_at, '[)')
WITH &&)
WHERE (status <> 'cancelled');                      -- violation: 23P01

-- 5. Upsert
INSERT INTO patients (email, name) VALUES ('a@b.com', 'An')
ON CONFLICT (email) DO NOTHING RETURNING id;

-- 6. Serializable (retry on 40001)
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT count(*) FROM dentists WHERE on_call;
UPDATE dentists SET on_call = false WHERE id = 1;
COMMIT;

-- 7. Advisory lock (released at commit)
```

```
SELECT pg_advisory_xact_lock(1001, 3);

-- 8. Deadlock prevention: lock in a fixed order
SELECT * FROM accounts WHERE id IN (1, 2) ORDER BY id FOR UPDATE;
```

Which is strongest

- **Most general guarantee: Serializable.** It prevents every anomaly, but only for transactions that run at Serializable, and it requires retry logic on 40001.
- **Strongest for a specific rule: a constraint.** It holds for every code path, isolation level and manual psql session. For Dentacore's double booking, an exclusion constraint beats Serializable.

Retry on 40001 (serialization) and 40P01 (deadlock). **Return to the user** 23505 (unique) and 23P01 (exclusion): those are business conflicts.

Prisma: raw SQL, transactions and locks

Use Prisma Client for CRUD and keep raw SQL in a few named repository methods for what Prisma can't express.

Method	Use for	Returns
`$queryRaw`	`SELECT`, or anything with `RETURNING`	rows
`$executeRaw`	writes without `RETURNING`, advisory locks (void result)	affected row count
`$queryRawUnsafe` / `$executeRawUnsafe`	dynamic SQL strings	not injection-safe; pass values as `$1` params

```
const rows = await tx.$queryRaw<{ stock: number }[]>`
  UPDATE items SET stock = stock - 1
  WHERE id = ${itemId} AND stock >= 1
  RETURNING stock`;

await tx.$executeRaw`SELECT pg_advisory_xact_lock(1001, ${dentistId})`;

// Dynamic pieces
const statusFilter = status ? Prisma.sql`AND status = ${status}` :
Prisma.empty;
await prisma.$queryRaw`SELECT * FROM appointments WHERE id IN
(${Prisma.join(ids)}) ${statusFilter}`;
```

Gotchas: raw SQL uses database names (`starts_at` , quoted `"Appointment"`), not Prisma field names; `count(*)` returns `BigInt` (cast `::int`); some params need casts (`::uuid` , `::jsonb`); the `<T>` result type is not validated. Exclusion constraints go in a `prisma migrate dev --create-only` migration.

Mixing Prisma Client and raw SQL

- Prisma Client by default; raw SQL only with a one-line reason ("needs row lock").
- Raw SQL lives in repository methods, never controllers.
- Inside a transaction every call uses the transaction client. In Dentacore that is `txHost.tx` from `@nestjs-cls/transactional` ; a direct `PrismaService` call runs outside the transaction silently.
- Prisma owns all schema changes, including ones its schema language can't express.
- Cover raw queries and concurrency logic with integration tests against real Postgres.

`update` vs `updateMany` as a guard

Since Prisma 5, `update` accepts non-unique filters beside a unique field, so both work as an atomic guard:

	`updateMany`	`update`
No row matches	returns `{ count: 0 }`	throws P2025
Returns	count only	the updated record

	`updateMany`	`update`
`id` is `undefined`	updates every row matching the other filters	fails (unique field required)

Never pass `employee?.id`: an `undefined` filter is ignored by Prisma.

Does `update` lock the row?

Yes. Postgres row-locks every row an `UPDATE` modifies (`FOR NO KEY UPDATE` when no key column changes) until the transaction ends. It blocks other updates, deletes and `FOR UPDATE` on that row, but not plain `SELECT`s. A second guarded update waits, re-checks its `WHERE` after the first commits, and matches 0 rows. Confirm the generated SQL once with `log: ['query']`.

Invite flow: fixing the race

The write re-checks the condition itself: a guarded update on `inviteStatus = UNINVITED` and `deletedAt = null` makes a second concurrent invite match 0 rows and return 409.

The race without it: two staff invite the same email at once; both lookups see UNINVITED; both transactions mark INVITED and create a token, so the employee gets two invites.

Why moving the lookup into the transaction is not enough: a plain `SELECT` takes no lock, so under Read Committed both still read UNINVITED. It works only with Serializable plus retry, or a raw `FOR UPDATE`. The guarded update needs neither and leaves the lookup function unchanged.

Check	Purpose	Conditions	On failure
`findUninvitedByEmail`	Find and validate the employee by email	email (primary or secondary), not deleted, UNINVITED	404 (staff creates a pending user) or 409 if several match
Guarded update	Make sure nothing changed before writing	not deleted, UNINVITED, by id	409 already invited or removed

The lookup uses `findMany({ take: 2 })` and refuses ambiguous matches instead of guessing; enforcing email uniqueness across `primaryEmail` and `Email` is a later schema decision.

Current `inviteEmployee`

```
async inviteEmployee(email: string) {
  await this.txHost.withTransaction(async () => {
    const employeeRecords = await
this.employeeRepo.findUninvitedByEmail(email);
    if (employeeRecords.length > 1) throw new ConflictException('Users with
duplicate email found');
    const employee = employeeRecords[0];
    if (!employee) throw new NotFoundException('User not found');

    const invited = await this.employeeRepo.markInvited(employee.id);   //
guarded updateMany
    if (!invited) throw new ConflictException('Employee already invited or
removed');

    await this.outbox.add(this.txHost.tx, {
      topic: 'employee.invited',
      key: employee.id,
      payload: { employeeId: employee.id, email, issuerNationalId: 'request-
user-id-placeholder' },
    });
  });          // commits here
  this.outbox.notify();   // after commit
}
```

- `const employee = employeeRecords[0]` plus the `!employee` check is what narrows the type under `noUncheckedIndexedAccess`; use `employee.id` everywhere after it.
- Serializable is not needed: the guard works at the default Read Committed level.

`@Transactional()` and `notify()`

`@Transactional()` commits when the decorated method returns, so `notify()` inside it fires before the message is visible and the relay waits for its next 1 s poll. Either keep `withTransaction` as above, or split the method:

```
async inviteEmployee(email: string) {
  await this.markInvitedAndQueue(email);   // commits when this returns
  this.outbox.notify();
}

@Transactional()
private async markInvitedAndQueue(email: string) { /* lookup, guard,
outbox.add */ }
```

Never stack both wrappers: the inner `withTransaction` just joins the outer transaction. Rule: anything that must run after commit goes outside the decorated method.

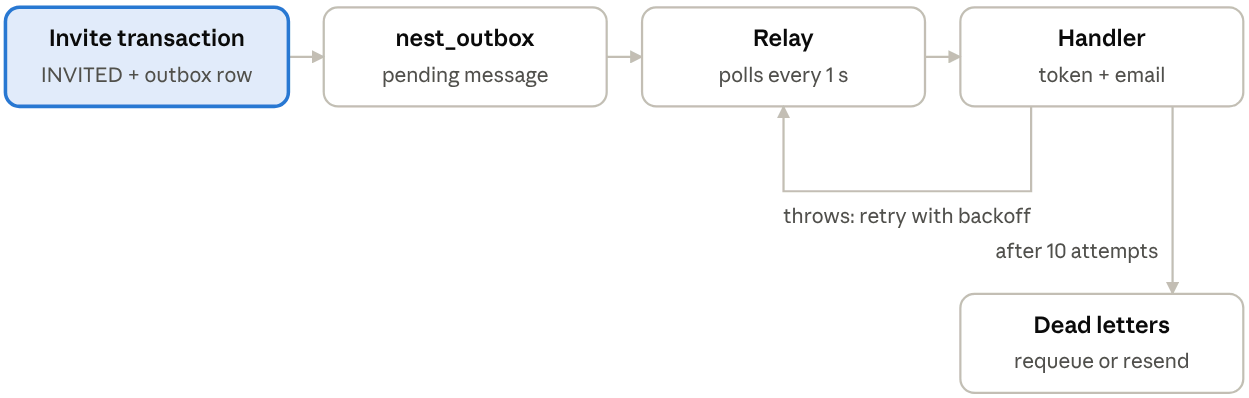
Transactional outbox with `@nestjs/outbox`

The email job is written to a Postgres table in the same transaction as the status change, so it exists if and only if the invite committed; a relay then delivers it to a handler and retries with backoff.

The dual-write problem it solves: committing and then calling the mail service (or publishing to a broker) can crash in between. The employee is INVITED, no email ever goes out, and the guarded update blocks a retry. Publishing first is worse: an email can go out for an invite that then rolls back.

What it guarantees: rollback means no email; commit means the email is attempted until it succeeds or is dead-lettered. That is eventual, at-least-once delivery, not a transaction across Postgres and mail.

The email job commits with the invite, then retries until it is sent



invite email via the outbox · 5 parts, 1 retry loop

Only the invite transaction is atomic; everything to its right happens later, retried until the email is sent or dead-lettered.

Setup in Dentacore

1. **Driver adapter.** `fromPrisma` needs a Prisma client built with `@prisma/adapter-pg`: `super({ adapter: new PrismaPg({ connectionString: process.env.DATABASE_URL }) })` in `PrismaService`. That is the only place `DATABASE_URL` is read; the store reuses that connection.
2. **Store and module** in `AppModule`:

```
OutboxModule.forRoot({
  relay: { pollInterval: '1s' },
  retry: { attempts: 10, backoff: { delay: '1s', maxDelay: '1m' } },
}),
// providers:
{
  provide: PostgresOutboxStore,
  inject: [PrismaService, OutboxStorage],
  useFactory: (prisma: PrismaService, storage: OutboxStorage) =>
    new PostgresOutboxStore({ executor: fromPrisma(prisma) }, storage),
},
```

3. **Tables.** They live in their own `nest_outbox` schema (switch the schema in Prisma Studio to see them); Prisma migrations never touch it. In development they are created at startup. The CLI does not read `.env`, so pass the URL:

```
npx @nestjs/outbox status  --url
"postgres://dentacore:dentacore@localhost:5435/dentacore"
npx @nestjs/outbox migrate --url
"postgres://dentacore:dentacore@localhost:5435/dentacore"
```

4. **Producer.** `outbox.add(this.txHost.tx, { topic, key, payload })` inside the transaction, `outbox.notify()` after commit (see the invite flow above). The array form of `$transaction` can't carry `outbox.add()`.
5. **Handler.** Registered as a provider in `InvitationModule`; it creates the token and sends the email (see the token section):

```
@OnOutboxMessage('employee.invited', { consumer: 'invite-email' })   //
consumer name must stay stable
async handleInvite(msg: InvitePayload, ctx:
OutboxHandlerContext<Prisma.TransactionClient>) {
  await this.txHost.withTransaction({ timeout: 15_000 }, async () =>
    ctx.processInTransaction(this.txHost.tx, () =>
this.invitationService.issueInvite(msg)),
  );
}
```

`processInTransaction` records the message id in the same transaction as the token insert, so duplicates are skipped. If the email throws, the token insert rolls back and the message is retried. The relay runs outside HTTP, so if you see "no active CLS context", add `@UseCls()` to the handler.

6. **Dead letters.** `OutboxAdminController` (in `outbox-admin/`) exposes `GET admin/outbox/dead-letters` and `POST .../:id/requeue` through `OutboxDeadLetters`, behind an admin guard.

Checking it works

What you see in `nest_outbox`	Meaning
Message processed within 1–2 s, `InviteToken` row appears	Handler running
Pending with 0 attempts	Handler not registered as a provider
Attempt count rising	Handler throws: check the logs

Outbox vs RabbitMQ

The outbox gets a message reliably out of your database; RabbitMQ routes messages between services. No broker is needed for Dentacore today: the local transport runs the handler in-process.

	`@nestjs/outbox`	RabbitMQ
What it is	Library implementing a pattern	Standalone broker server

	`@nestjs/outbox`	RabbitMQ
Where messages live	Your Postgres (`nest_outbox`)	RabbitMQ queues
Joins your DB transaction	Yes, its whole purpose	No: publishing is a separate network call
Retries and dead letters	Built in	Assembled from retry queues, TTLs, DLX
Duplicate protection	Built-in inbox	Consumer implements it
Routing	To a handler or one transport	Exchanges, topics, fan-out
Throughput	Bounded by Postgres polling	High volume, low latency

When several services need the event later, route outbox messages through `ClientProxyTransport` to RabbitMQ with publisher confirms enabled.

Invite token security

An invite token works like a temporary password, so the database stores only its SHA-256 hash, and the raw value exists only in the handler's memory and in the email.

Why the handler creates the token

With the outbox, the email is sent later by the handler, which shares no memory with `inviteEmployee`. If `inviteEmployee` generated the token, the raw value would have to travel in the outbox message, which is a row in Postgres, and a database leak would expose working tokens. Generating it in the handler avoids that, and has two side benefits:

- The 7 days start when the email actually goes out, not when it was queued.
- A failed send rolls back the token insert, leaving no orphan tokens.

SHA-256 is enough here because a 32-byte random token can't be guessed; bcrypt is for human-chosen passwords.

```
generate() { return randomBytes(32).toString('base64url'); }           //
goes in the email
digest(raw: string) { return createHash('sha256').update(raw).digest('hex');
}   // goes in the DB
```

Where the token travels

Location	Verdict	Why
Request body	Use this	Not in access logs, proxy logs or history; same treatment as the password
Query or path param	Avoid	Logged by servers and proxies, kept in history, leaks via `Referer`
Header	Works, wrong fit	Headers carry per-request credentials; this is one form submission

The email link uses a fragment, `http://localhost:3000/invite/accept#token=...`: browsers never send the part after `#` to the server or in `Referer`. The accept page reads it, removes it with `history.replaceState`, keeps it in memory, and loads no analytics scripts.

Verify endpoint

`POST /invitation/verify` with `{ inviteToken }` lets the page say "this link has expired" before the employee types a password. It is UX, not security:

- Read only: never sets `consumedAt`.
- Returns `{ valid: true }` and nothing else (no email, role or id).
- One error for every failure: "Invalid or expired link".
- Shares `findActiveInviteToken` with accept, so both agree on "valid".
- Rate-limited with `@nestjs/throttler`, and excluded from `ApiKeyMiddleware` (a key the browser sends is public).

Accept (`setNewPassword`, still to finish)

1. Check the two passwords match, then hash the password before opening a transaction (bcrypt/argon2 take ~100 ms).

2. In one transaction: consume the token with a guarded `updateMany` (`consumedAt: null, revokedAt: null`, require count 1).
3. Set `passwordHash` and move `inviteStatus` from INVITED to ACTIVE with the status guard.
4. Revoke the employee's other unused tokens, which also kills duplicate links from a retried email.

Beyond two systems: Redis, sagas and compensation

No transaction spans processes or databases; Postgres stays the source of truth, and every other system becomes an outbox-driven follow-up with its own retries.

Can a transaction run across services?

- **NestJS service classes in one app:** yes. They share the transaction through `txHost.tx` (or a `tx` parameter); any call through `PrismaService` directly escapes it.
- **Separate processes, databases or external APIs:** no. Two-phase commit (`PREPARE TRANSACTION`) exists but needs a coordinator, holds locks and is unsupported by most providers. Use the outbox, idempotent consumers and compensation instead.

Adding Redis (or any third system)

Queue one message per follow-up in the same transaction; each topic has its own handler and retries independently:

```
await this.outbox.add(this.txHost.tx, [
  { topic: 'employee.invited', key: employee.id, payload: { /* ... */ } },
  { topic: 'employee.cache.invalidate', key: employee.id, payload: {
employeeId: employee.id } },
]);
```

- **Redis as a cache** (rebuildable): usually no outbox. Delete the key after commit and set a TTL.
- **Redis holding state that matters** (revoked sessions, rate limits): yes, through the outbox.
- Never make Redis the source of truth for data that must agree with Postgres. Watch outbox lag and dead letters per topic.

Sagas: steps that depend on each other

Example: booking with a deposit across Postgres, a payment provider and the dentist's calendar. Each step is a local transaction that records its success and queues the next step together; a permanent failure later triggers compensating actions.

```
@OnOutboxMessage('appointment.requested', { consumer: 'appointment-payment'
})
async chargeDeposit(msg: { appointmentId: string }, ctx:
OutboxHandlerContext<Prisma.TransactionClient>) {
  await this.txHost.withTransaction(async () =>
    ctx.processInTransaction(this.txHost.tx, async () => {
      const appt = await this.appointmentRepo.findById(msg.appointmentId);
      if (appt?.status !== 'PENDING_PAYMENT') return;             //
duplicate or cancelled

      const result = await this.payments.charge({
        amount: appt.depositAmount,
        idempotencyKey: `deposit-${appt.id}`,                      // retry
can't charge twice
      });

      if (result.declined) {                                       //
business failure: compensate
        await this.appointmentRepo.setStatus(appt.id, 'CANCELLED');
        await this.outbox.add(this.txHost.tx, { topic:
'appointment.cancelled', key: appt.id, payload: { appointmentId: appt.id }
});
        return;
      }
      await this.appointmentRepo.setStatus(appt.id, 'PAID');
      await this.outbox.add(this.txHost.tx, { topic: 'appointment.paid', key:
appt.id, payload: { appointmentId: appt.id, chargeId: result.chargeId } });
    }),
  );
}
```

Step	Compensation if a later step fails permanently
Create appointment	Set to CANCELLED (keep the history)
Charge deposit	Refund at the provider, its own idempotency key

Step	Compensation if a later step fails permanently
Calendar event	Nothing if it was the step that failed

Rules:

1. A status column records where the saga is; every handler checks the status it expects first.
2. Transient failures throw and retry; business failures start compensation; exhausted retries go to dead letters for a human.
3. Every external call carries an idempotency key.
4. Irreversible steps (emails) go last.
5. A compensation is not a perfect undo: a refund still shows on the statement.

This chain is choreography (each handler queues the next). Past three or so steps, orchestration (one state machine deciding each step) is easier to follow.

Code review findings and TODOs

From the review of the repository-layer refactor (7 Nov 2026). Ticked items were shown fixed in the conversation; the rest are still open.

Invite flow

- [x] ~~`inviteEmployee` treated the `findMany` array as one record (risk: invite every uninvited employee)~~
- [x] ~~`markInvited` not awaited, so the 409 guard never fired~~
- [x] ~~`@Transactional()` stacked with `withTransaction`; `notify()` ran before commit~~
- [ ] Register `InviteEmailHandler` in `InvitationModule` providers (without it no email is ever sent)
- [ ] Add `@UseCls()` to `handleInvite` if the relay reports no CLS context; drop its unused `PrismaService`
- [ ] Pass the real `issuerNationalId` from the authenticated user instead of the placeholder
- [ ] Finish `setNewPassword`: guarded token consume, password + ACTIVE status, revoke other tokens

- [ ] Add a staff "Resend invite" action: revoke unused tokens, queue a new `employee.invited`

Modules and DTOs

- [ ] Remove `InvitationModule` from its own `imports`
- [ ] Export `EmployeeRepository` from `EmployeeModule` and `TokenService` from `CryptoModule` instead of re-providing them
- [ ] `InviteEmployeeDto`: add `class-validator` decorators, drop `inviteToken`; delete the empty `create-invite-token.dto.ts`
- [ ] `CreateUserDto`: drop `rawPassword`; guard `@Transform(... toLowerCase())` against non-strings; stop lowercasing names

Schema

- [ ] Make `Employee.passwordHash` nullable for uninvited employees
- [ ] Rename `deleteAt` to `deletedAt` on Address, Phone, InviteToken, Email, Role and Permission
- [ ] Make `InviteToken.employeeId` required; add a unique index on `tokenHash`; add the `ACTIVE` invite status
- [ ] Decide on email uniqueness across `primaryEmail` and `Email`

Architecture

- [ ] Repositories return data or domain errors; services map them to HTTP exceptions
- [ ] Check that `P2002`'s `meta.target` is populated with the pg driver adapter
- [ ] Route every DB call through `txHost.tx`; remove direct `PrismaService` use in services
- [ ] Exclude the public invite routes (`verify`, `accept`) from `ApiKeyMiddleware` and rate-limit them
- [ ] Integration tests: parallel invites (one wins), outbox retry after a forced mail failure